Universidad de La Laguna

Introduction to Design Patterns

Academic Year 2025–2026

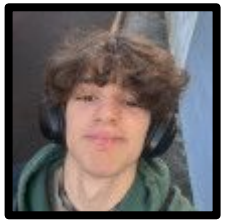
Introduction to Design Patterns

Link to code examples in these slides
Link to all code examples in this course

Don't forget to update the repos!

Contact us!

Kyrylo Chvanov (kyrylo.chvanov.20@ull.edu.es)



Adriel Reyes Suárez (adriel.reyes.suarez.33@ull.edu.es)

F. Gabriel Ruiz Ruiz (gabriel.ruiz.40@ull.edu.es)

Index

- What is a design pattern?
- Some background information
- Why should you be interested?
- How to use design patterns
- Classification of design patterns
- Builder pattern
- Facade pattern

Index

- Singleton pattern
- Adapter pattern
- Mediator pattern
- References

What is a design pattern?

What is a design pattern?

“Design patterns are typical solutions to common problems in software design.”

What is a design pattern?

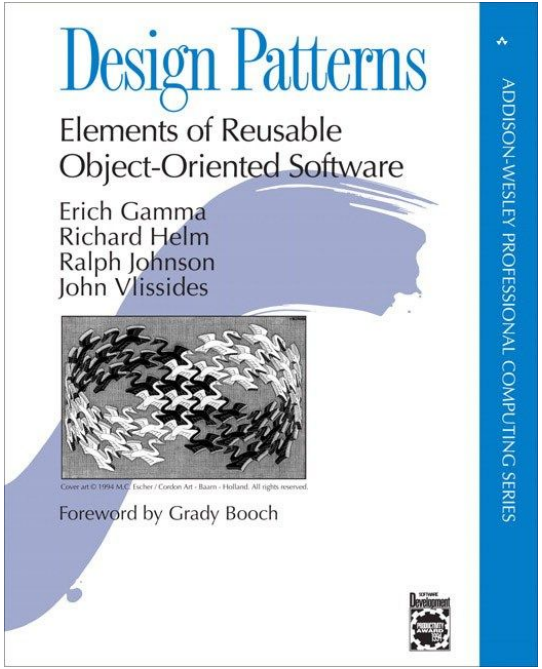
“Design patterns are typical solutions to common problems in software design.”

Source: Refactoring Guru

Some background information

Design Patterns

Gang of Four (GOF)



[More info here!]

Why should you be interested?

Why should you be interested?

PAI 2526 / Práctica ?? - ??? / 2025-2026 Cuestionario ??. Introduction to Design Patterns

2025-2026 Cuestionario ??. Introduction to Design Patterns

Por hacer: Recibir una calificación

Abre: ???

Cierra: ???

Why should you be interested?

PAI 2526 / Práctica ?? - ??? / 2025-2026 Cuestionario ??. Introduction to Design Patterns

2025-2026 Cuestionario ??. Introduction to Design Patterns

Por hacer: Recibir una calificación

Abre: ???

Cierra: ???

That's a good reason, but there are better ones...

Why are they important?

Why are they important?

- A Universal Vocabulary

Why are they important?

- A Universal Vocabulary
- Tested, Bulletproof Solutions

Why are they important?

- A Universal Vocabulary
- Tested, Bulletproof Solutions
- Better Code Maintainability

Why are they important?

- A Universal Vocabulary
- Tested, Bulletproof Solutions
- Better Code Maintainability
- Faster Development Time

How to use them

It is NOT about copying and pasting…

How to use them

It is NOT about copying and pasting…

…rather **computational thinking!**

There are a lot of them...

There are a lot of them...

There are 23 classic design patterns...

There are a lot of them…

There are 23 classic design patterns…

…but that is just the tip of the iceberg!

There are a lot of them…

We will only cover the easiest to understand patterns!

Classification

Classification

- Creational

Classification

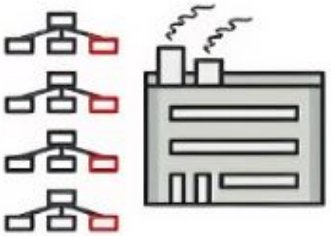
- Creational
- Structural

Classification

- Creational
- Structural
- Behavioural

Creational

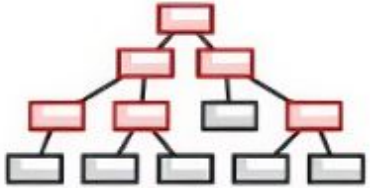
Defines how objects are created.



More about creational patterns…

Structural

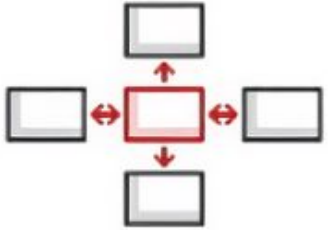
Defines how objects relate to each other.



More about structural patterns…

Behavioral

Define how objects communicate with each other.



More about behavioural patterns...

Are design patterns related?

Are design patterns related?

OF COURSE!

Are design patterns related?

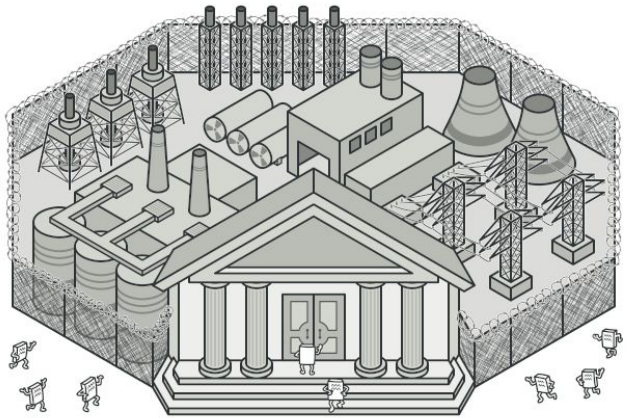
OF COURSE!

Check out their relations!

Facade pattern

Intent

Hides a lot of complexity behind a single, simplified method or interface.



Problem

- Requires executing different objects in a specific, strict order.

Problem

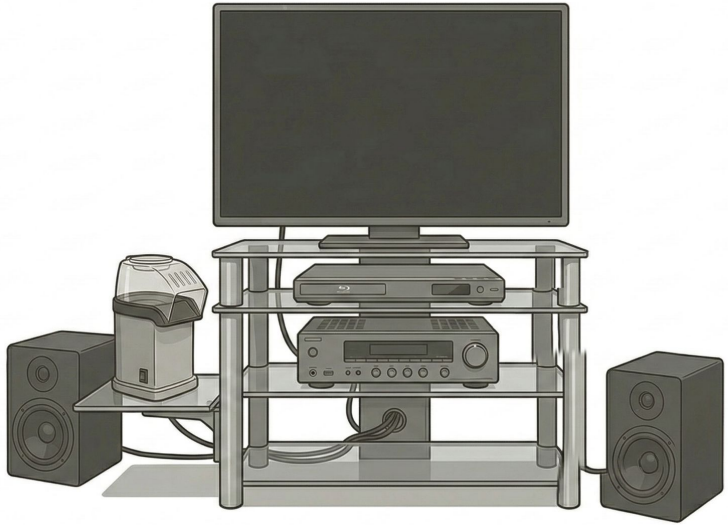
- Requires executing different objects in a specific, strict order.
- The client code becomes directly tied to this complex logic.

Problem

- Requires executing different objects in a specific, strict order.
- The client code becomes directly tied to this complex logic.
- Changes in object definitions break the code.

Problem

Imagine we want to watch a movie in a modern home theater with multiple devices.



Problem

```
// Instances initialized here...
popcornMaker.turnOn();
television.turnOn();
lights.dim();
// …
```

Problem

facade-home-theater-problem.ts

Solution

- Centralize the setup logic into a single Facade class.

Solution

- Centralize the setup logic into a single Facade class.
- Expose straightforward methods (e.g., watchMovie()).

Solution

- Centralize the setup logic into a single Facade class.
- Expose straightforward methods (e.g. watchMovie()).
- The client code remains short, clean, and completely decoupled.

Solution

```
class HomeTheaterFacade {
  constructor(private readonly amp: Amplifier, …) {}

  watchMovie(): void {
    this.popcornMaker.turnOn();
    this.lights.dim();
    // …
  }
}
```

Solution

```
function main() {
  // Initialize subsystem components (amp, bluray, lights…)
  const homeTheater = new HomeTheaterFacade(amp, bluray, …);
  homeTheater.watchMovie();
  homeTheater.endMovie();
}
```

facade-home-theater-solution.ts

In a nutshell…

In a nutshell...

- **Intent**: Hides system complexity behind a single, simplified interface.

In a nutshell...

- **Intent**: Hides system complexity behind a single, simplified interface.
- **Problem**: Client code becomes tied to complex logic and breaks easily when objects change.

In a nutshell...

- **Intent**: Hides system complexity behind a single, simplified interface.
- **Problem**: Client code becomes tied to complex logic and breaks easily when objects change.
- **Solution**: Centralize setup logic into one class with straightforward methods.

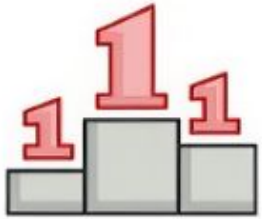
In a nutshell…

- **Intent**: Hides system complexity behind a single, simplified interface.
- **Problem**: Client code becomes tied to complex logic and breaks easily when objects change.
- **Solution**: Centralize setup logic into one class with straightforward methods.
- **Benefit**: Keeps client code short, clean, and decoupled from the subsystem.

In a nutshell...

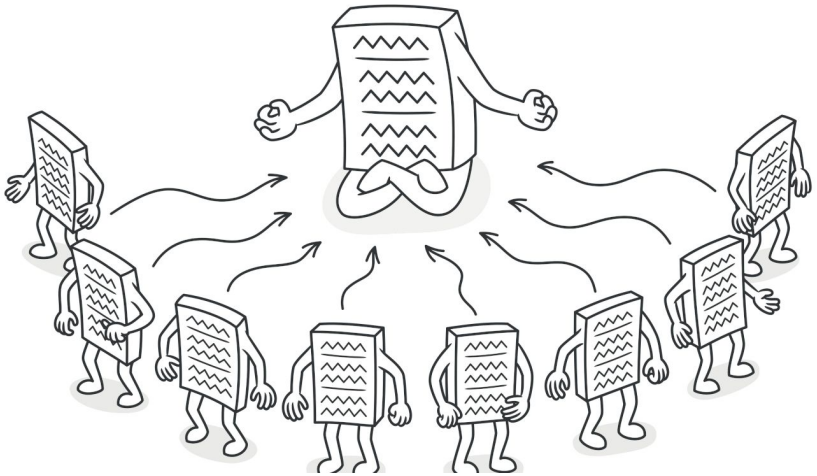
- **Intent**: Hides system complexity behind a single, simplified interface.
- **Problem**: Client code becomes tied to complex logic and breaks easily when objects change.
- **Solution**: Centralize setup logic into one class with straightforward methods.
- **Benefit**: Keeps client code short, clean, and decoupled from the subsystem.
- **Risk**: The Facade can grow too large and become a "God Class".

Singleton pattern



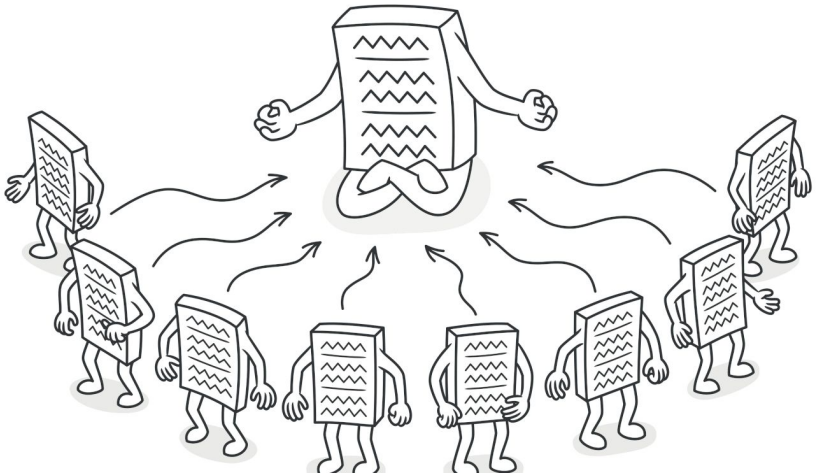
Intent

- Ensures a class has only one single instance.



Intent

- Ensures a class has only one single instance.
- Provides a global point of access to that instance.



Problem

Imagine we want to represent a Government entity, of which, logically, there should only be one.

Problem

```
class Government {
  constructor(private leader: string) {}
}
```

[singleton-pattern-government-problem.ts]

Problem

Unrestricted instantiation:

Problem

Unrestricted instantiation:

```
const ministryOfHealth = new Government('Alice');
const ministryOfDefense = new Government('Alice');


console.log(ministryOfHealth === ministryOfDefense);
```

Problem

Standard global variables:

Problem

Standard global variables:

```
let officialGovernment = new Government(‘Alice’);


officialGovernment = new Government(‘Bob’);
```

Solution

- Make the constructor private to completely disable the new operator.

Solution

- Make the constructor private to completely disable the new operator.

- Expose a static method that acts as a secure, read-only global window to the cached object.

Solution

```
class Government {
  private static instance: Government;
  private constructor() {
    // Initialization logic goes here...
  }
  // ... (global access point in next slide)
}
```

[singleton-pattern-government-solution.ts]

Solution

```
class Government {
  // ... (private constructor from previous slide)
  static getInstance(): Government {
    if (!Government.instance) {
      // Lazy initialization: only created when needed
      Government.instance = new Government();
    }
    return Government.instance;
  }
}
```

Solution

```
// Both variables point to the exact same memory address
const ministryOfHealth = Government.getInstance();
const ministryOfDefense = Government.getInstance();

// If one reference mutates the shared state...
ministryOfDefense.electNewLeader('President Bob');

// ...the other reference immediately reflects the change
ministryOfHealth.passLaw('New Budget'); // Led by President Bob
```

Disadvantages

- Unrestricted instantiation creates fragmented memory states.

Disadvantages

- Unrestricted instantiation creates fragmented memory states.
- Standard global variables are mutable and unsafe.

Disadvantages

- Unrestricted instantiation creates fragmented memory states.
- Standard global variables are mutable and unsafe.
- It solves two problems at once, violating the *Single Responsibility Principle* (SRP).

In a nutshell…

In a nutshell...

- **Intent**: Ensures a class has only one instance .

In a nutshell...

- **Intent**: Ensures a class has only one instance .
- **Problem**: Unrestricted instantiation and standard global variables are unsafe.

In a nutshell...

- **Intent**: Ensures a class has only one instance .
- **Problem**: Unrestricted instantiation and standard global variables are unsafe.
- **Solution**: Make the constructor private to disable the new operator and use a static method for access.

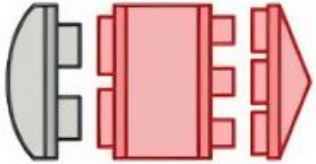
In a nutshell...

- **Intent**: Ensures a class has only one instance .
- **Problem**: Unrestricted instantiation and standard global variables are unsafe.
- **Solution**: Make the constructor private to disable the new operator and use a static method for access.
- **Benefit**: Provides a global, controlled point of access to that specific instance.

In a nutshell...

- **Intent**: Ensures a class has only one instance.
- **Problem**: Unrestricted instantiation and standard global variables are unsafe.
- **Solution**: Make the constructor private to disable the new operator and use a static method for access.
- **Benefit**: Provides a global, controlled point of access to that specific instance.
- **Risk**: Violates the Single Responsibility Principle (SRP) by managing both its lifecycle and its business logic.

Adapter pattern



Intent

Enables objects with incompatible interfaces to collaborate.

Problem

We have designed a product for our program:

```
// Target: interface our program understands
class Product {
  constructor(protected readonly priceEuros: number) { }
  // ...
}
```

Problem

With this getter:

```
// ...
getPriceEuros(): number {
  return this.priceEuros;
}
// ...
```

Problem

We have created a function that uses it:

```
function printPrice(product: Product) {
  console.log(`Price: ${product.getPriceEuros()} €`);
}
```

Problem

We now need to use an external library, but it uses Dollars instead of Euros!

Problem

```
// Adaptee: the library we need to use but is incompatible!
class ExternalService {
  getAmountInDollars(): number {
    return 10.50; // $10.50
  }
}
```

Problem

Our printer does NOT understand it...

```
const service = new ExternalService();
printPrice(service);
// Error: service isn't a Product and we can't change this!
```

adapter-pattern-money-problem.ts

Solution

We need a class that:

Solution

We need a class that:

- Implements the target (Product).

Solution

We need a class that:

- Implements the target (Product).
- Wraps the external incompatible library (ExternalService).

Solution

We need a class that:

- Implements the target (Product).
- Wraps the external incompatible library (ExternalService).
- Translates incompatible data.

Solution

These should be the changes needed:

```
const service = new ExternalService();
const adapter = new MoneyAdapter(service);
printPrice(adapter);
// Now this works!
```

Solution

This method call works wonders:

```
function printPrice(product: Product) {
  console.log(`Price: ${product.getPriceEuros() } €`);
}
```

Solution

Adapter (MoneyAdapter) inherits from the target class (Product).

```
class MoneyAdapter extends Product {
  // ...
}
```

adapter-pattern-money-solution.ts

Solution

Creates an instance using the adaptee (ExternalService):

```
// ...
constructor(private externalService: ExternalService) {
  super(0); // This value will be ignored later on...
}
// ...
```

Solution

Overrides the target's method:

```
// ...
getPriceEuros(): number {
  const dollars = this.externalService.getAmountInUSD();
  return dollars * 0.92; // Conversion rate USD -> Euros
}
// ...
```

In a nutshell…

In a nutshell...

- **Intent**: Enables objects with incompatible interfaces to collaborate by acting as a wrapper.

In a nutshell…

- **Intent**: Enables objects with incompatible interfaces to collaborate by acting as a wrapper.
- **Problem**: Usage of an external service your code doesn’t understand.

In a nutshell...

- **Intent**: Enables objects with incompatible interfaces to collaborate by acting as a wrapper.
- **Problem**: Usage of an external service your code doesn't understand.
- **Solution**: Class that implements the target interface, wraps the incompatible object, and translates the data between them.

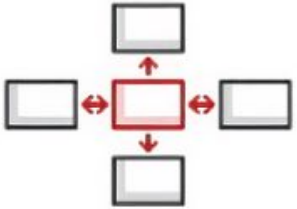
In a nutshell…

- **Intent**: Enables objects with incompatible interfaces to collaborate by acting as a wrapper.
- **Problem**: Usage of an external service your code doesn’t understand.
- **Solution**: Class that implements the target interface, wraps the incompatible object, and translates the data between them.
- **Benefit**: No need to change existing program logic.

In a nutshell…

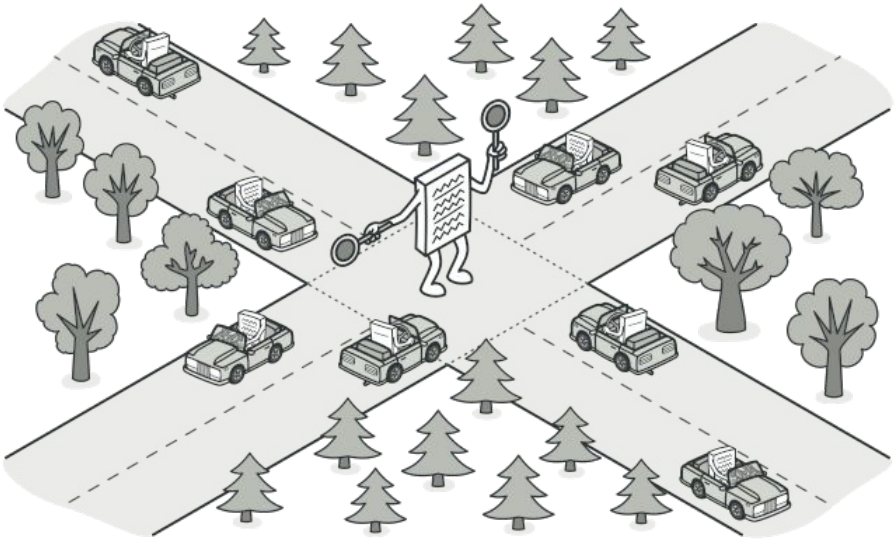
- **Intent**: Enables objects with incompatible interfaces to collaborate by acting as a wrapper.
- **Problem**: Usage of an external service your code doesn't understand.
- **Solution**: Class that implements the target interface, wraps the incompatible object, and translates the data between them.
- **Benefit**: No need to change existing program logic.
- **Risk**: Overall complexity of the code increases (more classes and interfaces required).

Mediator pattern



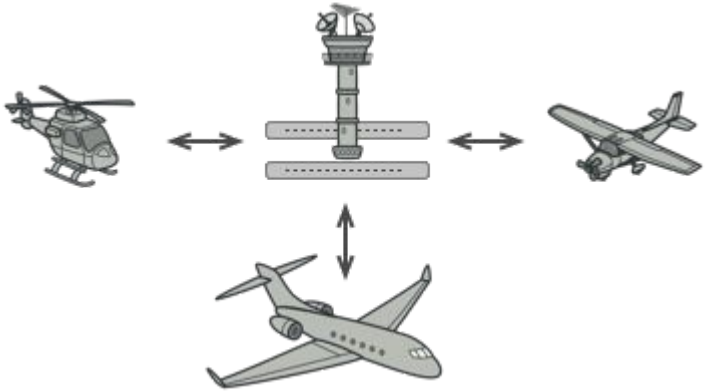
Intent

Reduce chaotic dependencies between objects.



Problem

Imagine we are managing an airport with multiple planes and a few runways.



Problem

Runway class:

Problem

Runway class:

```
class Runway {
  isClear(): boolean {
    return this.clear;
  }
}
```

Problem

Airplane class:

Problem

Airplane class:

```
class Airplane {
  land() {
    // …
  }
}
```

Problem

Who is landing where?

```
const planeOne = Airplane();
const planeTwo = Airplane();
const planeThree = Airplane();
// …
const runwayOne = Runway();
const runwayTwo = Runway();
const runwayThree = Runway();
```

Problem

```
const planeOne = Airplane();
const planeTwo = Airplane();
const planeThree = Airplane();
// …
const runwayOne = Runway();
const runwayTwo = Runway();
const runwayThree = Runway();
```

Problem

air-traffic-problem.ts

Solution

We need a mediator!

Solution

We need a mediator!

```
class AirTrafficTower {
  private runways: Runway[] = [];
  addRunway(runway: Runway): void {
    this.runways.push(runway);
  }
}
```

Solution

We need a mediator!

```
class AirTrafficTower {
    requestLanding(plane: Airplane): void {
      // …
   }
}
```

Solution

air-traffic-solution.ts

In a nutshell…

In a nutshell…

- **Intent:** Centralizes complex communications and control between interacting objects.

In a nutshell...

- **Intent:** Centralizes complex communications and control between interacting objects.
- **Problem:** Direct, many-to-many object interactions create tightly coupled and unmaintainable code webs.

In a nutshell...

- **Intent:** Centralizes complex communications and control between interacting objects.
- **Problem:** Direct, many-to-many object interactions create tightly coupled and unmaintainable code webs.
- **Solution:** Interposes a dedicated coordinator object to handle all routing and communication between components.

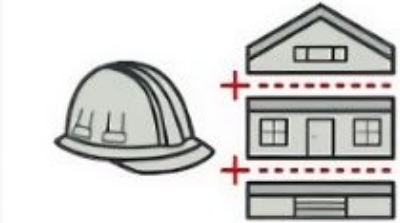
In a nutshell...

- **Intent:** Centralizes complex communications and control between interacting objects.
- **Problem:** Direct, many-to-many object interactions create tightly coupled and unmaintainable code webs.
- **Solution:** Interposes a dedicated coordinator object to handle all routing and communication between components.
- **Benefit:** Favors the Single Responsibility Principle and Open/Closed Principle.

In a nutshell…

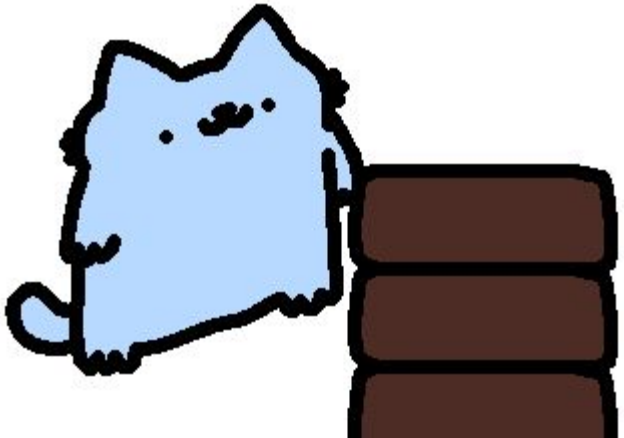
- **Intent:** Centralizes complex communications and control between interacting objects.
- **Problem:** Direct, many-to-many object interactions create tightly coupled and unmaintainable code webs.
- **Solution:** Interposes a dedicated coordinator object to handle all routing and communication between components.
- **Benefit:** Favors the Single Responsibility Principle and Open/Closed Principle.
- **Risk:** Over time a mediator can evolve into a God Class.

Builder pattern



Intent

- Enables you to build complex objects step by step.



Intent

- We can get different representations of an object...

Intent

- We can get different representations of an object…

…with the same code!

Problem

Complex objects require large constructors…

Problem

Complex objects require large constructors...

Initializing them can be a hassle!

Problem

Complex objects require large constructors...

Initializing them can be a hassle!

Read up on Telescoping Constructors!

Problem

```
class User {
  constructor(private name: string, private dni?: number,
      private address?: string, private: city?: string) {
    // ...
  }
  // ...
}
```

Problem

```
// I know the name and dni - constructor seems fine.
const jorge = new User('Jorge', 10000);


// I only know the name and city...
const abel = new User('Abel', 0, '', 'Sta. Cruz');
// This is not comfortable to work with!
```

builder-pattern-user-problem.ts

Solution

Delegate the object construction to another object: **builders**!

Solution

We are aiming to achieve this:

```
const userBuilder = new UserBuilder(); // the builder!
const abel: User =
    userBuilder.setName('Abel').setCity('Sta. Cruz').build();
// No need to initialize unknown variables!
```

builder-pattern-user-solution.ts

Building standardization

Building standardization

What if we wanted to create a standard for creating default users…

Building standardization

What if we wanted to create a standard for creating default users...

...and works with different builders?

Building standardization

```
const redUserBuilder = new redUserBuilder();
const red1: User = redUserBuilder.setName('x').setCity('LL').build();
const blueUserBuilder = new blueUserBuilder();
const blue1: User = blueUserBuilder.setName('x').setCity('LL').build();
// ...
```

DRY - Don’t Repeat Yourself!

Building standardization

Let's create an object that follows the same recipe for any* given builder: **the director!**

**they must be related somehow (interface or abstract class).*

Building standardization

```
class UserDirector {
  buildDefaultUser(builder: AbstractBuilder): User {
    return builder.setName('x').setCity('LL').build();
  }
}
```

builder-pattern-user-director.ts

Building standardization

We are aiming to achieve this:

```
const director = new UserDirector();
const redUserBuilder = new redUserBuilder();
const red1: User = director.buildDefaultUser(redUserBuilder);
const blueUserBuilder = new blueUserBuilder();
const blue1: User = director.buildDefaultUser(blueUserBuilder);
// Director works for two different builders!
```

Building standardization

Why should you need directors?

In a nutshell…

In a nutshell...

- **Intent**: Enables the step-by-step construction of complex objects.

In a nutshell...

- **Intent**: Enables the step-by-step construction of complex objects.
- **Problem**: Complex objects often require large constructors (rough initialization).

In a nutshell…

- **Intent**: Enables the step-by-step construction of complex objects.
- **Problem**: Complex objects often require large constructors (rough initialization).
- **Solution**: Delegate object construction to specialized *builder* objects.

In a nutshell…

- **Intent**: Enables the step-by-step construction of complex objects.
- **Problem**: Complex objects often require large constructors (rough initialization).
- **Solution**: Delegate object construction to specialized *builder* objects.
- **Benefit**: Keeps client code clean by allowing it to only set necessary parameters.

In a nutshell…

- **Intent**: Enables the step-by-step construction of complex objects.
- **Problem**: Complex objects often require large constructors (rough initialization).
- **Solution**: Delegate object construction to specialized *builder* objects.
- **Benefit**: Keeps client code clean by allowing it to only set necessary parameters.
- **Risk**: Overall complexity of the code increases (more classes and interfaces required).

In a nutshell…

- **Intent**: Enables the step-by-step construction of complex objects.
- **Problem**: Complex objects often require large constructors (rough initialization).
- **Solution**: Delegate object construction to specialized *builder* objects.
- **Benefit**: Keeps client code clean by allowing it to only set necessary parameters.
- **Risk**: Overall complexity of the code increases (more classes and interfaces required).
- **Standardization:** use *director* objects to follow a specific build algorithm.

Quiz time!

Quiz time!

What is a **disadvantage** of the Builder Pattern?

Quiz time!

What is a **disadvantage** of the Builder Pattern?

a) Builders must be used with directors.

Quiz time!

What is a **disadvantage** of the Builder Pattern?

a) Builders must be used with directors.

b) It uses polymorphism…

Quiz time!

What is a **disadvantage** of the Builder Pattern?

a) Builders must be used with directors.

b) It uses polymorphism…

c) Violates the Single Responsibility Principle.

Quiz time!

What is a **disadvantage** of the Builder Pattern?

a) Builders must be used with directors.

b) It uses polymorphism…

c) Violates the Single Responsibility Principle.

d) Overall complexity of the code goes up.

Quiz time!

What is a **disadvantage** of the Builder Pattern?

a) Builders must be used with directors.

b) It uses polymorphism…

c) Violates the Single Responsibility Principle.

d) Overall complexity of the code goes up.

Quiz time!

Quiz time!

What is a potential disadvantage of the Mediator pattern?

Quiz time!

What is a potential disadvantage of the Mediator pattern?

a) Cannot handle multiple objects

Quiz time!

What is a potential disadvantage of the Mediator pattern?

a) Cannot handle multiple objects

b) Mediator breaks encapsulation

Quiz time!

What is a potential disadvantage of the Mediator pattern?

a) Cannot handle multiple objects

b) Mediator breaks encapsulation

c) Mediator can become too complex

Quiz time!

What is a potential disadvantage of the Mediator pattern?

a) Cannot handle multiple objects

b) Mediator breaks encapsulation

c) Mediator can become too complex

d) Prevents communication

Quiz time!

What is a potential disadvantage of the Mediator pattern?

a) Cannot handle multiple objects

b) Mediator breaks encapsulation

c) Mediator can become too complex

d) Prevents communication

Quiz time!

Quiz time!

What is **true** about the Facade and Adapter pattern?

Quiz time!

What is **true** about the Facade and Adapter pattern?

a) The first defines new interfaces, the latter tries to make an existing one usable.

Quiz time!

What is **true** about the Facade and Adapter pattern?

a) The first defines new interfaces, the latter tries to make an existing one usable.

b) Both are creational patterns.

Quiz time!

What is **true** about the Facade and Adapter pattern?

a) The first defines new interfaces, the latter tries to make an existing one usable.

b) Both are creational patterns.

c) Both work with many types of objects (usually).

Quiz time!

What is **true** about the Facade and Adapter pattern?

a) The first defines new interfaces, the latter tries to make an existing one usable.

b) Both are creational patterns.

c) Both work with many types of objects (usually).

d) Both run into the danger of being turned into god classes.

Quiz time!

What is **true** about the Facade and Adapter pattern?

a) The first defines new interfaces, the latter tries to make an existing one usable.

b) Both are creational patterns.

c) Both work with many types of objects (usually).

d) Both run into the danger of being turned into god classes.

Quiz time!

Quiz time!

What is a known **drawback** of the Singleton pattern?

Quiz time!

What is a known **drawback** of the Singleton pattern?

a) It violates the Single Responsibility Principle (SRP).

Quiz time!

What is a known **drawback** of the Singleton pattern?

a) It violates the Single Responsibility Principle (SRP).

b) It is a structural pattern and cannot be used to create objects.

Quiz time!

What is a known **drawback** of the Singleton pattern?

a) It violates the Single Responsibility Principle (SRP).

b) It is a structural pattern and cannot be used to create objects.

c) It forces the developer to use the new keyword every time.

Quiz time!

What is a known **drawback** of the Singleton pattern?

a) It violates the Single Responsibility Principle (SRP).

b) It is a structural pattern and cannot be used to create objects.

c) It forces the developer to use the new keyword every time.

d) It requires creating exactly three instances of the class.

Quiz time!

What is a known **drawback** of the Singleton pattern?

a) It violates the Single Responsibility Principle (SRP).

b) It is a structural pattern and cannot be used to create objects.

c) It forces the developer to use the new keyword every time.

d) It requires creating exactly three instances of the class.

Universidad de La Laguna

Which pattern is the best?

Universidad de La Laguna

The one that best solves your problem!

Universidad de La Laguna

Any questions?

References

- Refactoring.guru
- GeeksforGeeks
- Wikipedia
- Captain Debug's Blog
- Stack Overflow (Builder design pattern: Why do we need a Director?)

References

- C# Design Patterns (dofactory)
- Design Patterns by the GoF (Amazon)
- 10 Design Patterns Explained in 10 Minutes